

Reinforcement Learning: From Foundations to Deep Approaches

Summer Semester 2026, Homework 2

Prof. Georgia Chalvatzaki, Franziska Herbert, Sophie Lüth



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Use the provided Python scripts to solve the problems. We provided you with a zip file for the assignment project that includes the Python scripts (to solve), a bash script (to run experiments), and a yaml file (to create a conda environment). The structure of the project is similar to MushroomRL.

We encourage you to use Conda to create an environment that contains all packages necessary for solving the assignment. We provide you with a yaml file (environment.yaml) that you can use to set up the conda environment as follows:

```
conda env create --file=environment.yaml
```

This assignment will not be graded! The solutions will be discussed on Thursday, June 25, 2026.

2.1 Feature Construction and Function Approximation

Solve the Puddle World environment (Fig. 1) using Online Sarsa Lambda using various types of features. The Puddle World is a continuous grid-world. The state space is a two-dimensional position in $[0, 1]^2$. The action space is a set of pairs, $\{(0.0, 0.0), (-0.05, 0.0), (0.05, 0.0), (0.0, -0.05), (0.0, 0.05)\}$, representing moves in both dimensions. At a position (p_x, p_y) , the agent receives at each step a reward signal defined as:

$$r = -1 + -2 (\mathcal{N}(p_x, 0.3, 0.1) \cdot \mathcal{N}(p_y, 0.6, 0.03) + \mathcal{N}(p_x, 0.4, 0.03) \cdot \mathcal{N}(p_y, 0.5, 0.1) + \mathcal{N}(p_x, 0.8, 0.03) \cdot \mathcal{N}(p_y, 0.9, 0.1))$$

where $\mathcal{N}(p, \mu, \sigma) = e^{-\frac{(p-\mu)^2}{2\sigma^2}} / \sigma\sqrt{2\pi}$. Please refer to [1] and the mushroom implementation for more details.

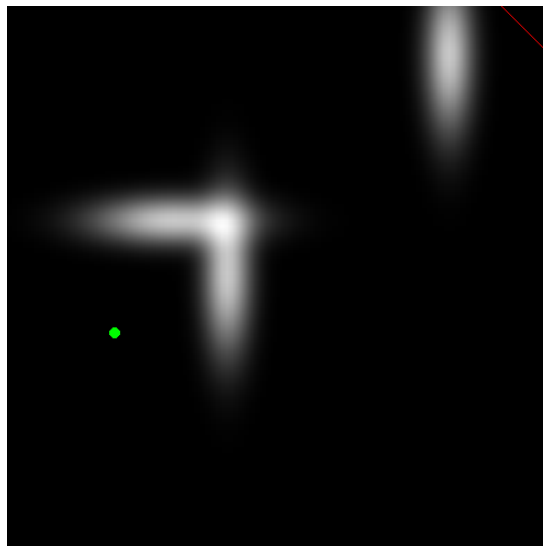


Fig. 1: A visual representation of the puddle world environment where the green point represents the agent, the bright region indicates the puddle, and the goal is considered after the red line.

Follow the `puddle_world_func_approx.py` template filling in the spots annotated by "[YOUR CODE!]"'. The task is composed of the following components:

1. Create an appropriate set of features to solve the Puddle World environment. The experiment file should support using Gaussian radial basis functions, Fourier basis, Rectangular Tiles, and raw state (no feature construction).
2. Create a Sarsa- λ agent with a linear approximator. Write down the training loop. Evaluate the agent and log the average return and episode length while training.
3. Tune the parameters of each feature annotated by "[TUNE PARAMETER!]". Run every feature type for 5 runs (more comparable: 30) and create two plots, comparing the average return and episode length for every feature type with the mean and confidence interval across all the 5 (30) runs.

2.2 Deep Q-Learning

DQN [2] is one of the earliest attempts to merge deep learning with Reinforcement Learning. In this exercise, we will study the performance of different DQN variants on the Asterix (Fig. 2) game from [3]. Instead of using a linear approximator as in 2.1, you will build your first ConvNet to approximate the Q function. Please use the hints in the Python script provided to implement the network.

Follow the *minatar_dqn_variants.py* template filling in the spots annotated by "[YOUR CODE!]". Some comments are included in the script to help you solve the task. Please use the values of the experiment's parameters specified in the script without changing them. The values of all the parameters used are in the *argparser.py* file.

In addition, use the *run_experiments.sh* bash script to run your experiments based on the algorithm's name passed to the script. For example, to run a DQN experiment, you can run the following command line:

```
sh run_experiment.sh DQN
```

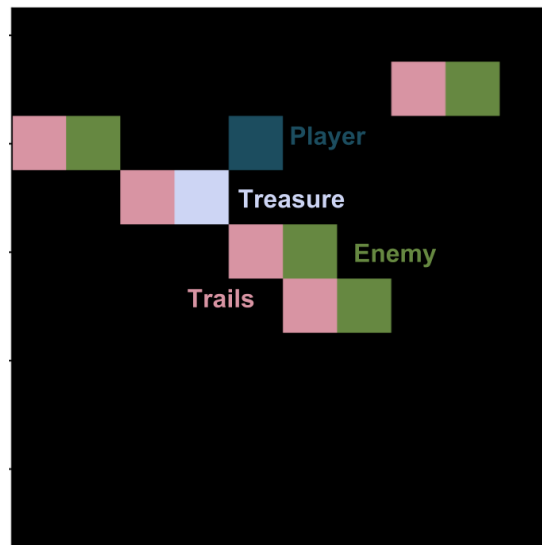


Fig. 2: Colour indicates active channel at each spatial location, but note that the representation provided by the environment consists of binary values for each channel and not RGB values.

The task is composed of the following components:

1. Use MushroomRL to implement the training and evaluation loop. Log the average return, value estimate at the initial states, and episode length metrics of the learning algorithms.
2. Implement a plotting script showing the learning data generated by the learning loop described in the previous question. Plots should show the mean and confidence interval of each algorithm computed from the different seeds.
3. Use MushroomRL to run each of the following algorithms for 2 (more comparable 5) runs on the Asterix game: DQN [2], DoubleDQN [4], MaxminDQN [5], and AveragedDQN [6] and the plot.

-
4. Implement a DQN algorithm without a replay buffer. This variant is the on-policy version of the original DQN algorithm. Train and evaluate the algorithm for 2 (more comparable 5) seeds and plot.
 5. In MaxminDQN, we use n models for each the online and target approximators. The estimation of Q is based on the minimum value across all the n models. Estimating the Q_{next} for the TD target, is similar to the DQN algorithm by taking the maximum of the Q values in the next state. In this question, we ask you to modify the MaxminDQN algorithm implemented in MushroomRL by creating a new algorithm called DoubleMaxminDQN where the Q_{next} is estimated based on the DoubleDQN algorithm. Train and evaluate the algorithm for 2 (more comparable 5) seeds and plot.
 6. Implement a Naive DQN algorithm without all the tricks introduced in the lecture. In addition to removing the replay buffer as in question 4, you need to estimate the Q_{next} using only the online approximation and ignore the clipping of reward. Train and evaluate the algorithm for 2 (more comparable 5) seeds and plot.

Reference

- [1] Thomas Degris, Martha White und Richard S. Sutton. “Off-Policy Actor-Critic”. In: *CoRR* abs/1205.4839 (2012). arXiv: [1205.4839](https://arxiv.org/abs/1205.4839). URL: <http://arxiv.org/abs/1205.4839>.
- [2] Volodymyr Mnih u. a. “Playing Atari with Deep Reinforcement Learning”. In: *CoRR* abs/1312.5602 (2013). arXiv: [1312.5602](https://arxiv.org/abs/1312.5602). URL: <http://arxiv.org/abs/1312.5602>.
- [3] Kenny Young und Tian Tian. “MinAtar: An Atari-inspired Testbed for More Efficient Reinforcement Learning Experiments”. In: *CoRR* abs/1903.03176 (2019). arXiv: [1903.03176](https://arxiv.org/abs/1903.03176). URL: <http://arxiv.org/abs/1903.03176>.
- [4] Hado van Hasselt, Arthur Guez und David Silver. “Deep Reinforcement Learning with Double Q-learning”. In: *CoRR* abs/1509.06461 (2015). arXiv: [1509.06461](https://arxiv.org/abs/1509.06461). URL: <http://arxiv.org/abs/1509.06461>.
- [5] Qingfeng Lan u. a. “Maxmin Q-learning: Controlling the Estimation Bias of Q-learning”. In: *CoRR* abs/2002.06487 (2020). arXiv: [2002.06487](https://arxiv.org/abs/2002.06487). URL: <https://arxiv.org/abs/2002.06487>.
- [6] Oron Anschel, Nir Baram und Nahum Shimkin. “Deep Reinforcement Learning with Averaged Target DQN”. In: *CoRR* abs/1611.01929 (2016). arXiv: [1611.01929](https://arxiv.org/abs/1611.01929). URL: <http://arxiv.org/abs/1611.01929>.